UNIVERSIDADE DE RIO VERDE - UniRV

FACULDADE DE ENGENHARIA DE SOFTWARE

CURSO DE BACHARELADO EM ENGENHARIA DE SOFTWARE

ESPECIFICAÇÃO DE REQUISITOS E PROJETO

DESENVOLVIMENTO DE UM EDITOR DE DESENHO VETORIAL INTERATIVO COM JAVAFX

Avaliação Prática N3 - Semestre 2026/1

Disciplina: Programação Orientada a Objetos 1

Docente: Prof. Me. Sergio Souza Novak

Rio Verde - GO, Junho de 2026

Sumário

1 Introdução e Objetivo Geral **1**

2 Arquitetura e Padrões de Projeto Exigidos **2**
2.1 State Pattern 2
2.2 Command e Memento (Undo/Redo) 2
2.3 Prototype 2
2.4 Composite 2

3 Requisitos Técnicos e Organização **3**
3.1 Estrutura de Pacotes 3
3.2 Encapsulamento 3

4 Funcionalidades Gráficas e Operacionais **4**
4.1 Painel de Desenho (Canvas) e Interfaces 4
4.2 Painéis Laterais e Status 4

5 Regras de Negócio e Comportamentos **5**
5.1 Validações Geométricas 5
5.2 Regras de Transformação Geométrica 5

6 Estrutura de Classes UML e Implementação **6**

7 Persistência, Tratamento de Erros e Avaliação **7**

Anexos: Telas do Sistema e Modelagem **8**

1 Introdução e Objetivo Geral

O desenvolvimento de sistemas modernos exige a aplicação de conceitos de programação orientada a objetos (POO), tais como abstração, polimorfismo, interfaces, herança e encapsulamento. O uso de padrões de projeto (Design Patterns) é crucial para garantir a manutenibilidade de interfaces gráficas.

Este documento apresenta a especificação para o projeto da avaliação N3 da disciplina de POO 1. Os alunos deverão implementar um **Editor de Desenho Vetorial Interativo** em Java utilizando **JavaFX**.

O objetivo do projeto é construir um software desktop que permita a criação, edição e persistência de formas geométricas 2D sobre uma tela interativa, aplicando padrões clássicos da GoF e a separação arquitetural MVC.

2 Arquitetura e Padrões de Projeto Exigidos

A arquitetura do sistema deve ser modular. Os alunos devem seguir a modelagem do diagrama de classes fornecido (`diagrama_classes.pdf`) e aplicar os padrões:

2.1 State Pattern

O comportamento da tela de desenho muda com a ferramenta ativa. Para evitar condicionais complexos, os alunos devem usar o padrão **State**:

- **Contexto**: A classe `EditorContext` gerencia o estado da aplicação.
- **Interface de Estado**: A interface `Ferramenta` expõe métodos como `aoPressionarMouse`, `aoArrastarMouse` e `aoClicar`.
- **Estados Concretos**: Classes como `FerramentaDesenhar`, `FerramentaSelecionar`, `FerramentaTransformar` e `FerramentaEditarVertice`.

2.2 Command e Memento (Undo/Redo)

O histórico de ações (Desfazer/Refazer) deve ser controlado pelo `HistoricoController`.

- O método `registrarEstado` deve salvar cópias profundas das formas.
- Os métodos `desfazer` e `refazer` controlam as pilhas de estados.

2.3 Prototype

A interface `FormaGeometrica` exige o método `clonar()` para permitir a duplicação de formas sem compartilhamento de referências em memória.

2.4 Composite

O agrupamento de formas sob um `groupId` comum deve se comportar como uma única forma lógica para fins de translação e transformação geométrica.

3 Requisitos Técnicos e Organização

Os requisitos definem a tecnologia e restrições obrigatórias do projeto.

Tabela 1: Matriz de Requisitos Técnicos

ID	Requisito	Especificação Técnica
RT01	Linguagem	Java (Versão 17 LTS ou superior)
RT02	Interface Gráfica	JavaFX 17+ com FXML ou código programático
RT03	Gerenciador	Apache Maven (`pom.xml`)
RT04	Persistência	Serialização nativa, JSON ou XML
RT05	Patterns	Implementação de State, Prototype e Memento
RT06	Arquitetura	Camadas separadas: Models, Controller e Views

3.1 Estrutura de Pacotes

O código fonte do projeto deve estar dividido nos seguintes pacotes:

- `org.example.Models`: Modelos geométricos (`FormaGeometrica`, `FormaBase`, `Ponto`, `Poligono`, `Circulo`).
- `org.example.Controller`: Controle e lógica de ferramentas (`EditorContext`, `HistoricoController` e ferramentas).
- `org.example.Views`: Telas e componentes (`ContainerApp`, `BarraLateralView`).

3.2 Encapsulamento

Os atributos das formas devem ser privados ou protegidos, com acesso mediado por *getters* e *setters*. O acoplamento entre Views e Controllers deve ser minimizado.

4 Funcionalidades Gráficas e Operacionais

O sistema deve apresentar um painel funcional dividido em três áreas principais:

4.1 Painel de Desenho (Canvas) e Interfaces

A área central interativa para renderização gráfica:

- **Criação Dinâmica**: Desenho por cliques (polígonos) ou arrasto (círculos e quadrados).
- **Snap to Grid**: Arredondamento de coordenadas do cursor para pontos da grade.
- **Seleção Visual**: Destaque visual nas formas selecionadas.

4.2 Painéis Laterais e Status

- **Barra Lateral**: Controles de toggle de ferramentas, cores e gerenciamento de camadas.
- **Barra de Status**: Coordenadas, total de formas e dados de área/perímetro em tempo real.

5 Regras de Negócio e Comportamentos

As classes do pacote `org.example.Models` devem seguir as regras:

5.1 Validações Geométricas

1. **Polígono Mínimo**: Um `Poligono` deve ter no mínimo 3 pontos.
2. **Círculo Válido**: Um `Circulo` deve ter raio estritamente positivo ($r > 0$).
3. **Formas Regulares**: Quadrados e Hexágonos calculam seus vértices automaticamente.
4. **Área e Perímetro**:
   - A área de polígonos segue a fórmula de Gauss:

$$A = \frac{1}{2} \left| \sum_{i=1}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) + (x_n y_1 - x_1 y_n) \right| \tag{1}$$

   - O perímetro do polígono é a soma das distâncias euclidianas.

5.2 Regras de Transformação Geométrica

1. **Translação**: Adiciona fatores de deslocamento dx e dy a cada ponto.
2. **Escala**: Multiplica a distância de cada ponto ao centroide pelo fator.
3. **Rotação**: Rotaciona os pontos em torno do centroide por um ângulo em radianos.

6 Estrutura de Classes UML e Implementação

A implementação deve espelhar a estrutura descrita no diagrama de classes.

```java
package org.example.Models;
import java.util.List;

public interface FormaGeometrica {
    void adicionarPonto(Ponto p);
    List<Ponto> getPontos();
    int getQuantidadePontos();
    String getCorPreenchimentoHex();
    void setCorPreenchimentoHex(String hex);
    String getCorBordaHex();
    void setCorBordaHex(String hex);
    double getEspessuraBorda();
    void setEspessuraBorda(double espessura);
    String getGroupId();
    void setGroupId(String groupId);
    double calcularArea();
    double calcularPerimetro();
    Ponto getCentroide();
    boolean contemPonto(double px, double py);
    void transladar(double dx, double dy);
    void escalar(double fator);
    void rotacionar(double anguloRadianos);
    void cisalhar(double shx, double shy);
    FormaGeometrica clonar();
}
```

Listing 1: Contrato para formas geométricas

7 Persistência, Tratamento de Erros e Avaliação

O software deve gerenciar a persistência em arquivos e prever o tratamento correto de falhas de entrada e saída.

Protocolo de Persistência: Os arquivos do editor vetorial devem ser exportados em formato estruturado (JSON ou XML), contendo as configurações do canvas e a lista de formas geométricas com seus atributos de estilo, coordenadas e identificadores de grupo.

Tratamento de Exceções: O sistema deve prever validações com blocos `try-catch` para tratar arquivos corrompidos (exibindo alerta gráfico explicativo sem interromper a aplicação) e alertar o usuário sobre alterações não salvas antes de fechar o editor.

Critérios de Avaliação: A composição da nota final considerará os seguintes critérios:

Critério de Avaliação	**Peso na Nota**
Modelagem Geométrica e Validações Matemáticas	25%
Implementação de Padrões de Projeto (State, Prototype, Memento)	35%
Usabilidade da GUI em JavaFX e Controles do Mouse	20%
Persistência XML/JSON e Robustez do Tratamento de Erros	20%

Formato de Entrega: Submissão do link do repositório Git com o código Maven funcional, o arquivo descritivo `README.md` e o diagrama de classes UML em PDF, entregues no portal acadêmico da UniRV.

Anexos: Telas do Sistema e Modelagem

Abaixo estão as capturas de tela do sistema e diagramas UML, apresentadas em tamanho grande (uma por linha):

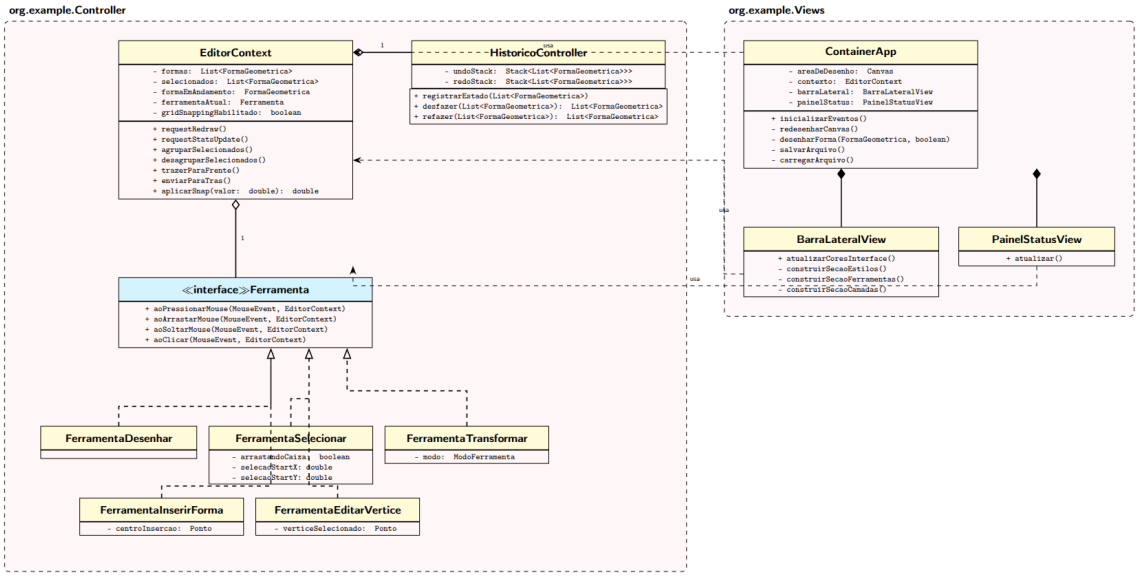


Figura 1: Diagrama de Classes: Controllers e Views.

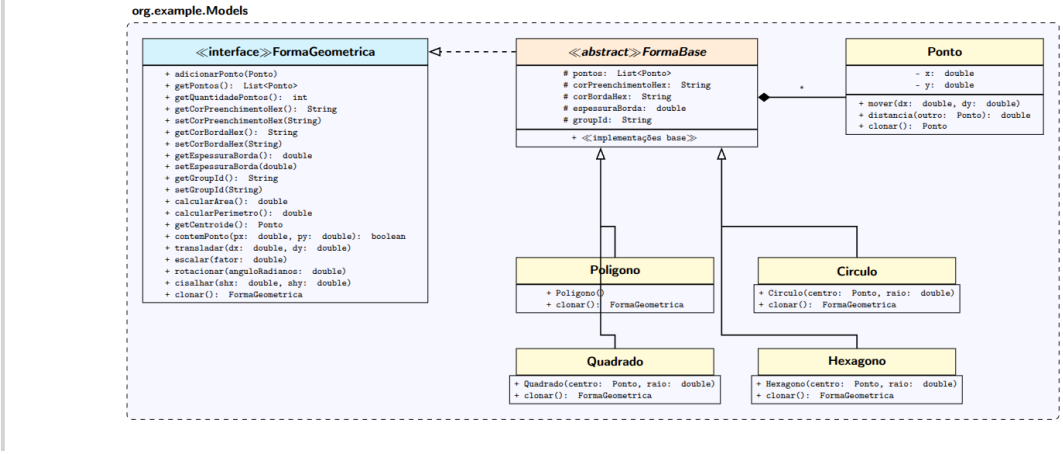


Figura 2: Diagrama de Classes: Models.

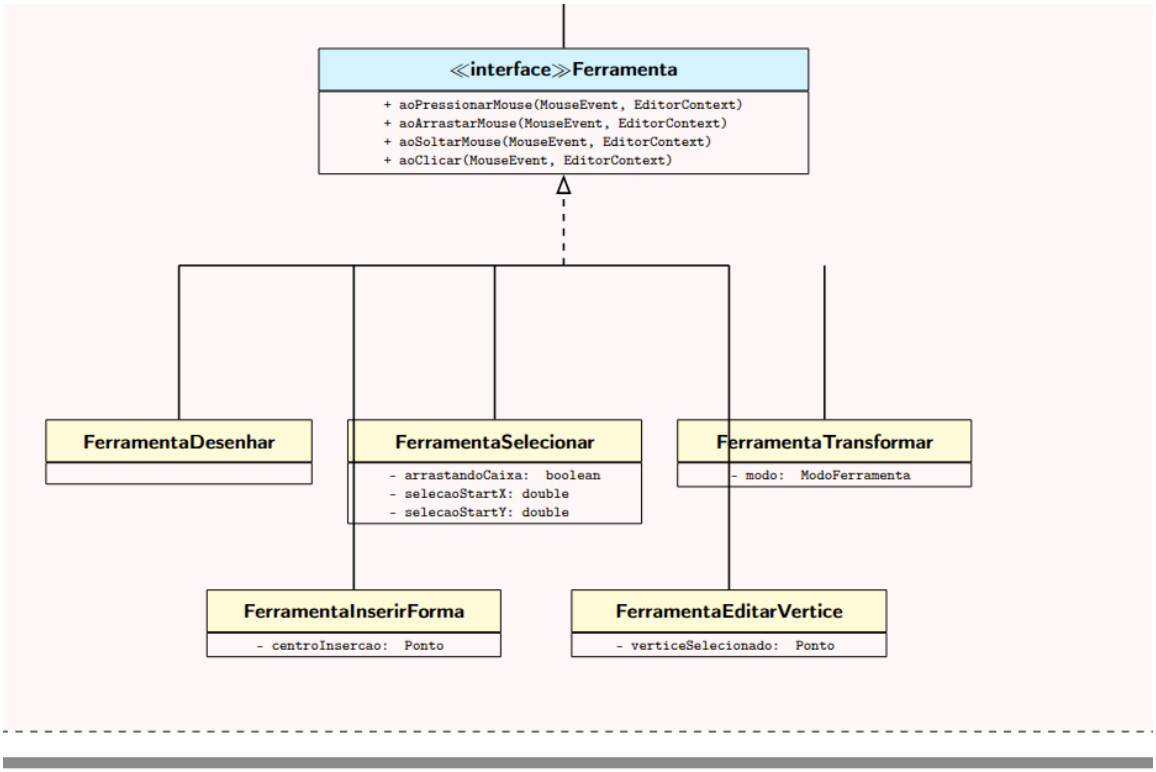


Figura 3: Diagrama de Classes: Ferramentas.

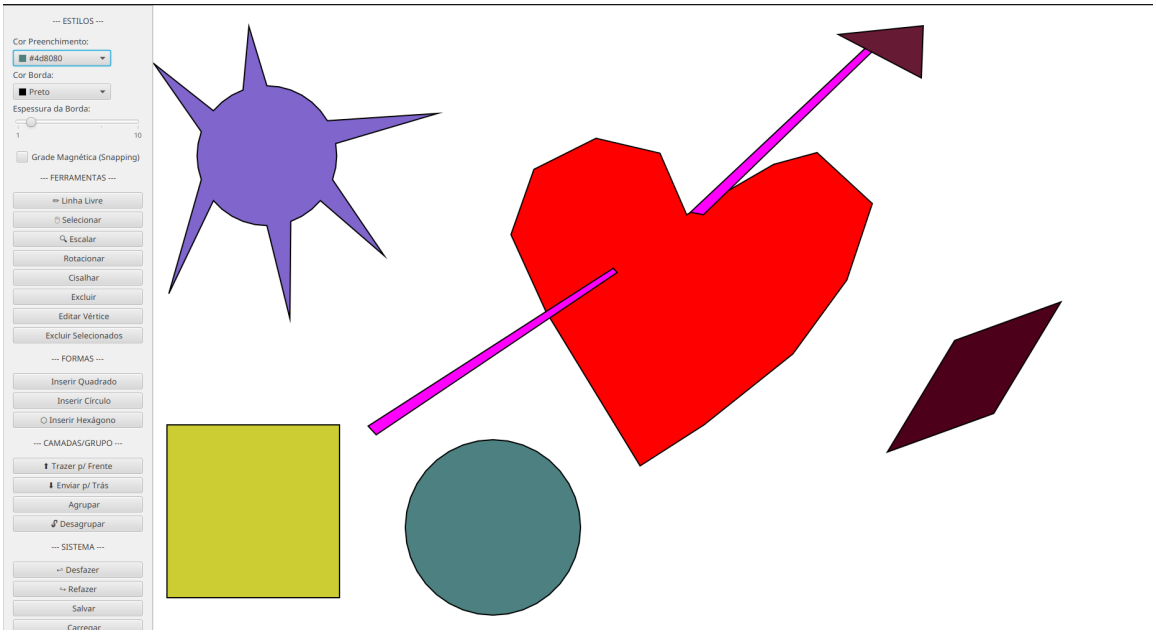


Figura 4: Interface do Sistema 1.

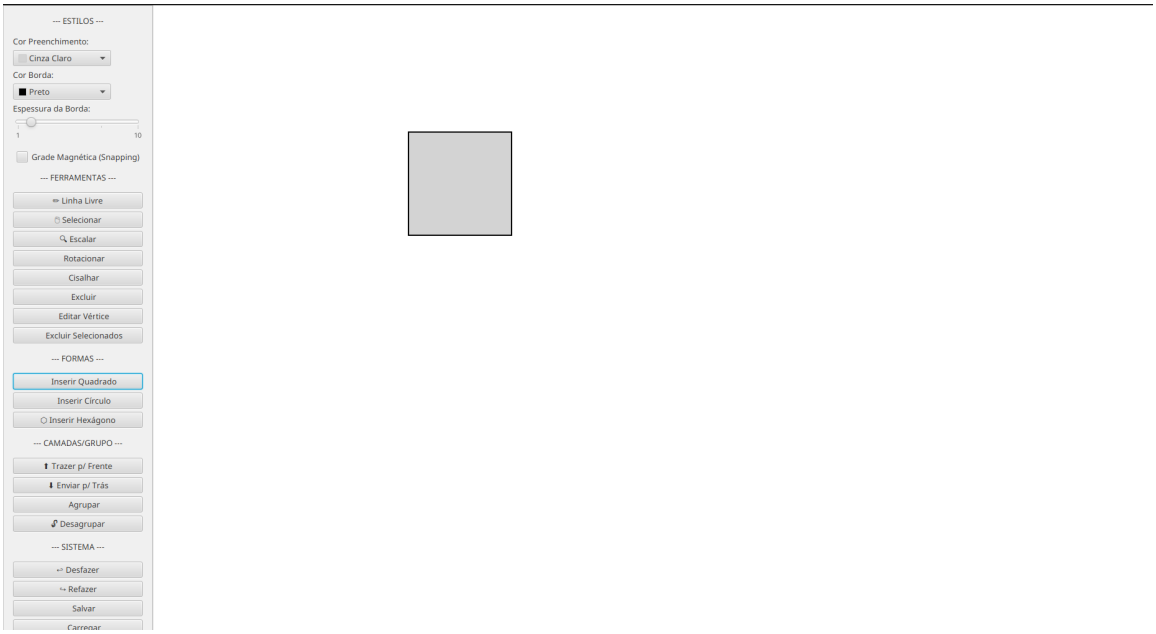


Figura 5: Interface do Sistema 2.

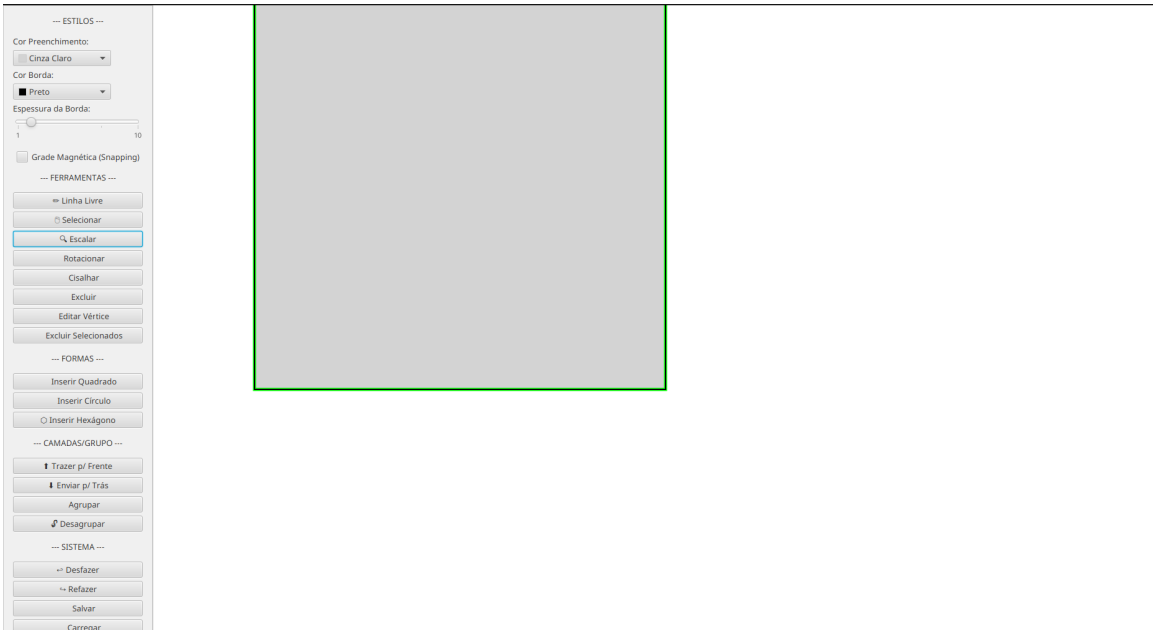


Figura 6: Interface do Sistema 3.

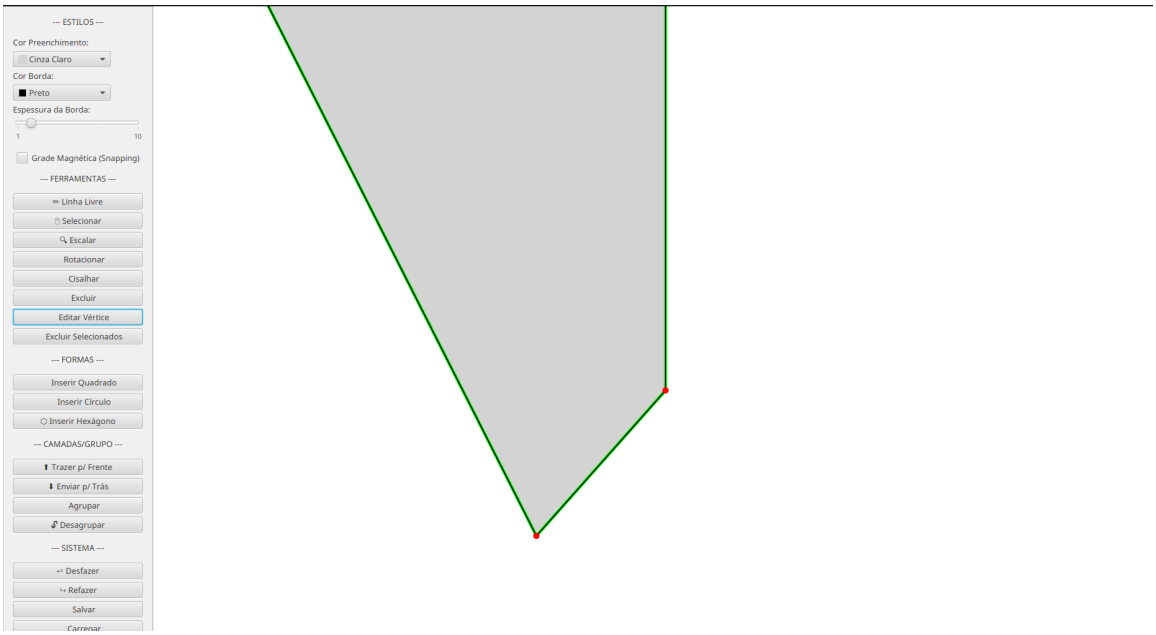


Figura 7: Interface do Sistema 4.

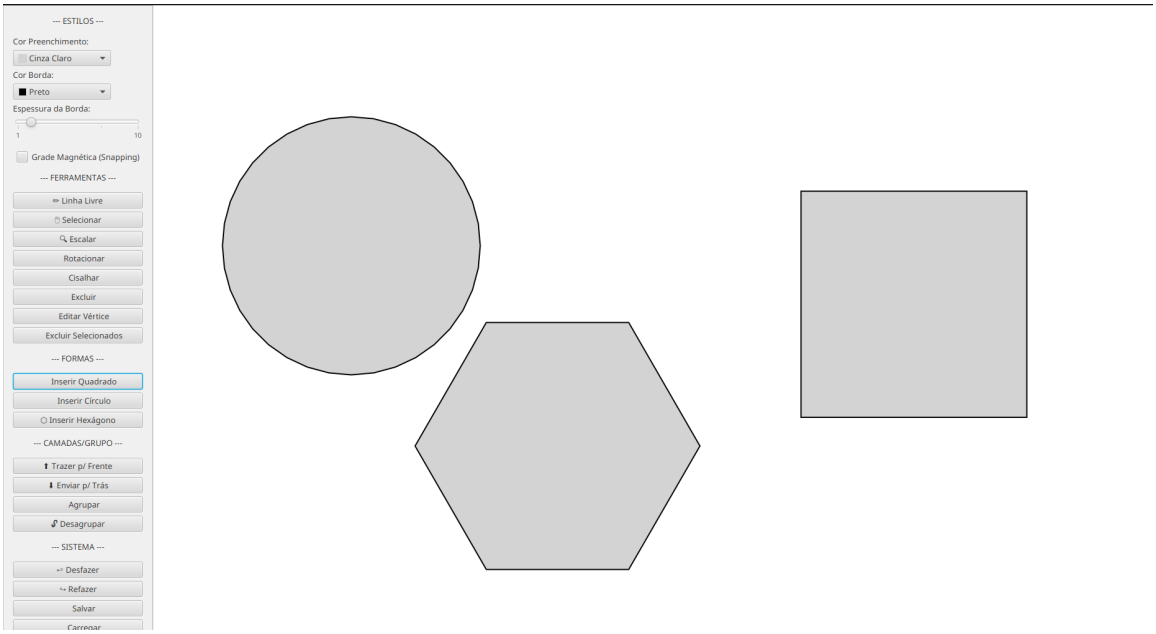


Figura 8: Interface do Sistema 5.

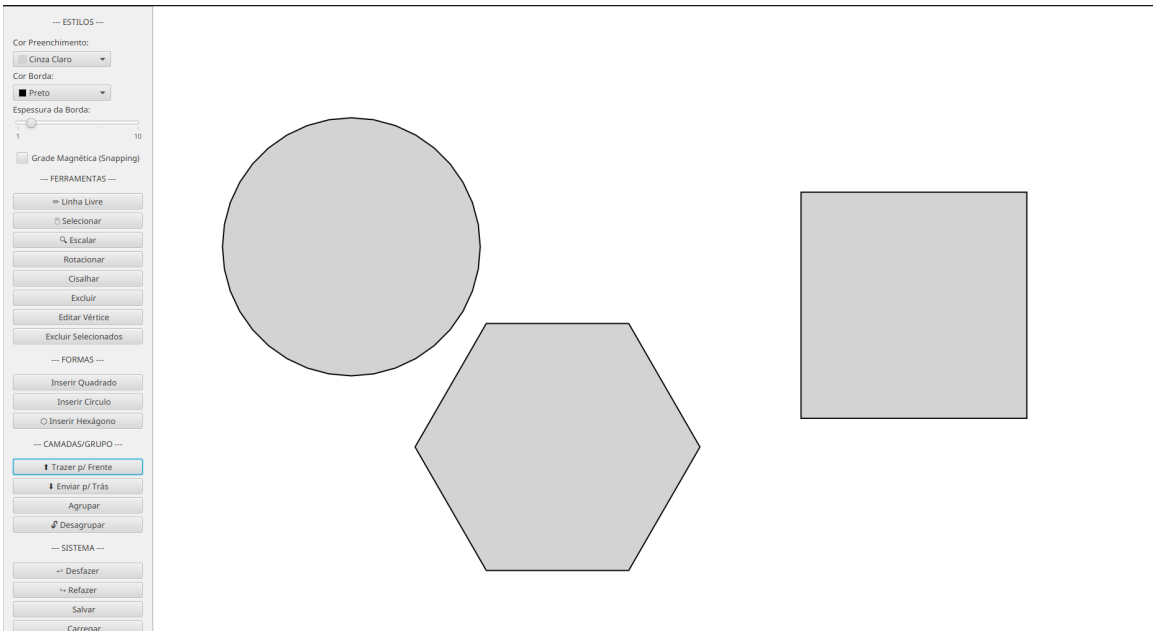


Figura 9: Interface do Sistema 6.

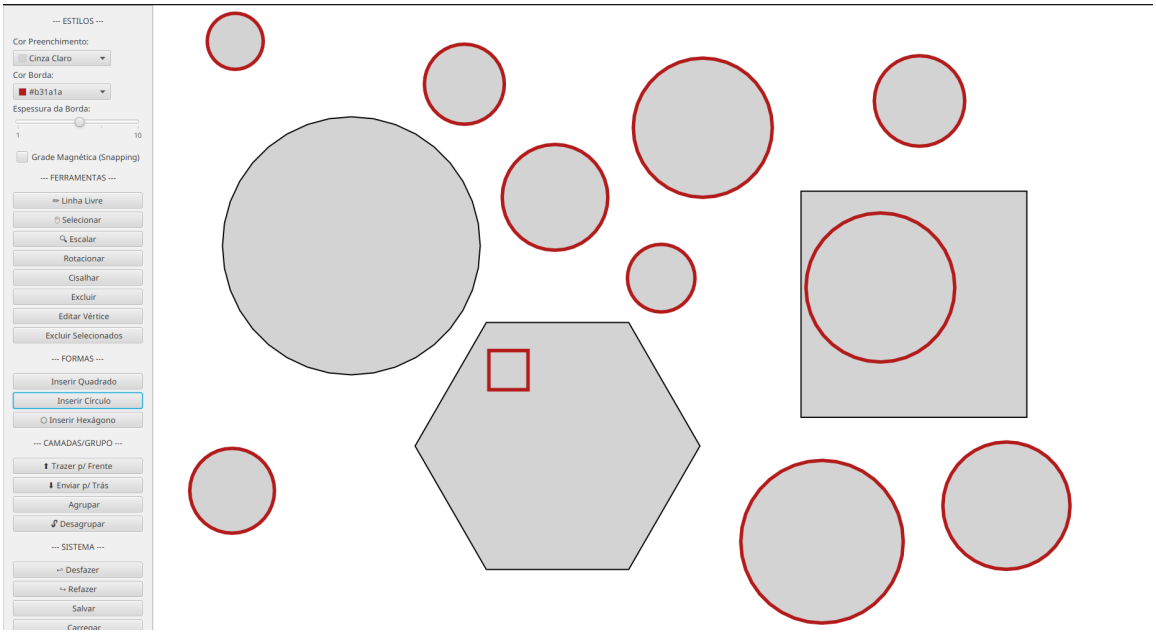


Figura 10: Interface do Sistema 7.